CS482/495/496 Software Project Proposal: Baltimore's Basketball Tournament

Chase Garnett, Hans van Lierop, Aidan Marshall, Manuel Mateo

2025-11-10

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name: Dr. Nguyen Ho
- Client title: Assistant Professor
- Client email address: tnho@loyola.edu
- Client employer: Loyola University Maryland
- How you know the client: Professor at our university

2 Project Description

2.1 Overview

[Add a few paragraphs describing your project succinctly. What problem are you trying to solve, what is the purpose of your project? Why does your client want this project?]

2.2 Key Features

[At this point you should have a basic understanding of your client's needs. List out the key features of the software system the client wants you to build.]

2.3 Why this Project is Interesting

[Why did you decide this project was interesting enough to you to be a capstone project? What about this project is enticing? Why should anyone care?]

2.4 Areas of CS required

[What subfields of computer science seem most likely to be relevant to your project? A capstone must involve multiple.]

2.5 Potential Concerns and Questions

[Is there any aspect of this project that makes you unsure if it will work, either due to your own interests/background, or that you aren't sure if it fits the requirements? Are there questions you have about this project that you want instructor feedback about?]

2.6 Summary of Efforts to Find a Project

(Not necessary for 482) [Briefly list out when/how you've discussed with this client, and if you've discussed with other clients who either didn't work out or didn't respond. If you considered a different project and it didn't work out, why didn't it work out?]

[Most CS495 projects end here. The sections below are for CS482 and CS496 software projects].

2.7 Comparison to Draft

[For CS496 only, focus on highlighting the major differences between the draft proposal in CS495 and this one here. If there are no major differences, you can remove this subsection.]

3 Requirements

3.1 Non-Functional Requirements

[Non-functional requirements are just as important as functional requirements. Dont forget to specify them.]

ID	NFR Title	Category	Description
NFR1	NFR Example 1	Usability	Description of the NFR (it does not follow a user story template)
NFR2	NFR Example 2	Security	Description of the NFR (it does not follow a user story template)

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

[In CS482, all functional requirements are written as User Stories. In CS496, some projects may use a different template to write the requirements. The table below is an example of writing the Stories. Adapt accordingly to different templates or if you want to display more info.]

ID	Story Title	Points	Description
S1	Story Example 1	5	As a user, I want to write a user story example, so that people will understand them.
S2	Story Example 2	2	As a user, I want to write a user story example, so that people will understand them.

Table 2: Functional requirements as User Stories.

4 System Design

4.1 Architecture

We will use an MVC architecture. As we understand them, the "view" is the actual interface of the website, the "model" is the backend or the data that the website runs on, and the "controller" is what connects them, handling input from the UI that needs to be put in the model and translating data from the model to a displayable format in the UI.
The main modules will be for:

- UI design (color, theme, page layout)
- Handling input from the UI (button, form, upload)
- Accessing and modifying database data (image upload/display, account info)

4.2 Diagrams

Sequence Diagram

4.3 Technology

- TypeScript - Programming language
- React - Frontend Library
- Material UI - Frontend Library
- Node.js - Runtime Engine
- Express.js - Backend Framework
- MongoDB - NoSQL Database
- Jest - Testing Framework

4.4 Coding Standards

- Contact the customer (Dr. Ho) if there are any uncertainties with requirements
- We are using standard React naming conventions: PascalCase for components, classes, and interfaces; camelCase for variables and functions; names starting with "use" for custom hooks; and UPPER_CASE for constants. All names should show a clear purpose (just name variables what they do).
- Commit small changes so we can go back if something breaks
- Only code with 60% branch coverage should be committed
- Add tests when bugs are found
- Pair program as much as possible (for learning and bug prevention)
- At least one other team member must accept a merge request
- We will update any documentation necessary as we change the codebase

4.5 Data

The application will utilize MongoDB, which aligns with the MERN stack. The core data structure is organized into several key sections. The users section stores all accounts, differentiating between roles (Admin, Manager, Guardian, Teen) and managing relationships. The tournaments section contains details for each competition, including rules, sponsor logos, and themes. Teams are managed in the teams section, linking them to their respective tournament, manager, and list of player members. Game schedules, scores, and live stream links are stored in the games section, while individual player performances are tracked per game in the playerStats section. Team-based communication is facilitated through the chat section.

The core information collected includes a unique username, email address, password, first and last name, and date of birth. Users will also have the option of uploading a profile picture. The system will assign each user a specific role: administrator, Manager, Guardian, or Teen. To establish critical relationships, the data model includes links between accounts; Guardian accounts will store a reference to the Teen accounts they represent, and Teen accounts will store a reference to their Guardian. Furthermore, user data will track team affiliations, with Managers storing a list of the teams they manage. This forms the key data that will drive our approach. How we form relationships and how we provide information to the users.

4.6 UI Mocks

Figure 1: Home Page

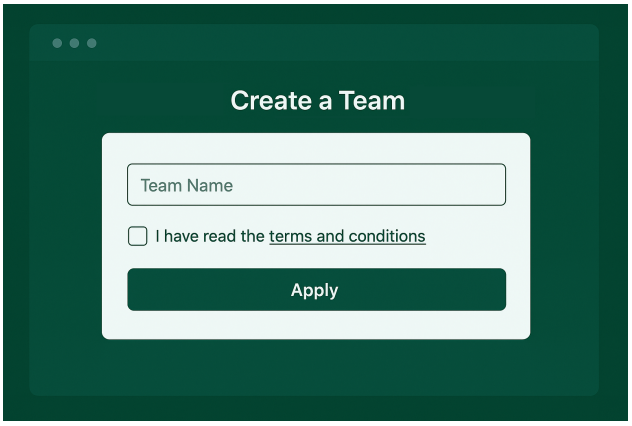


Figure 2: Team Creation

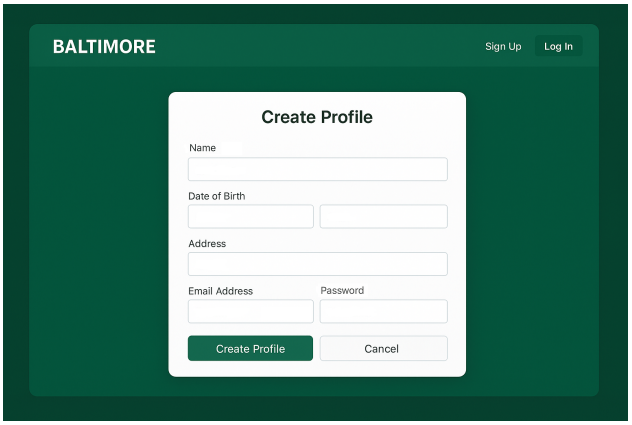


Figure 3: Profile Creation

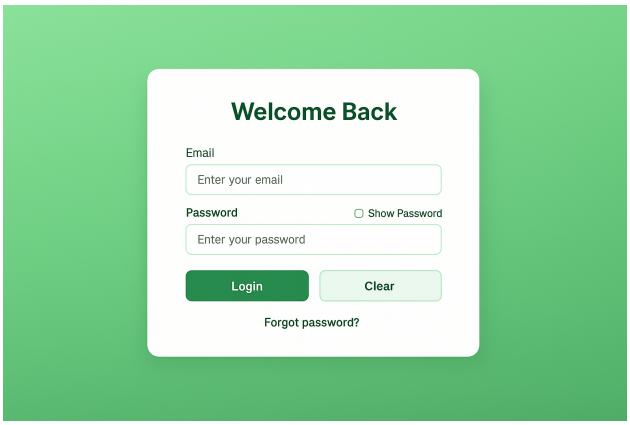


Figure 4: Login Page

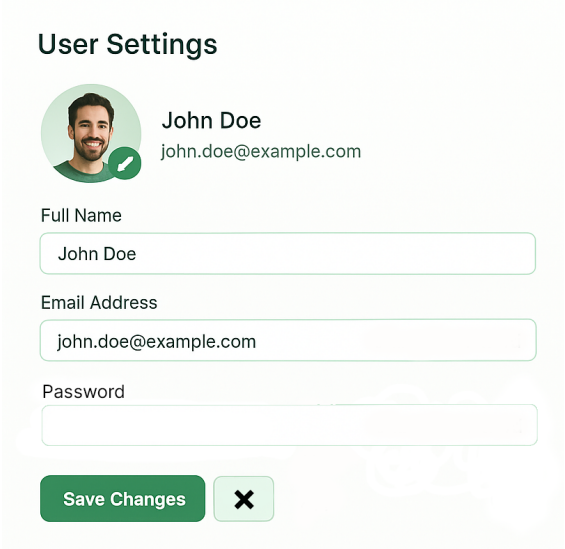


Figure 5: User Settings

Figure 6: Team Schedule

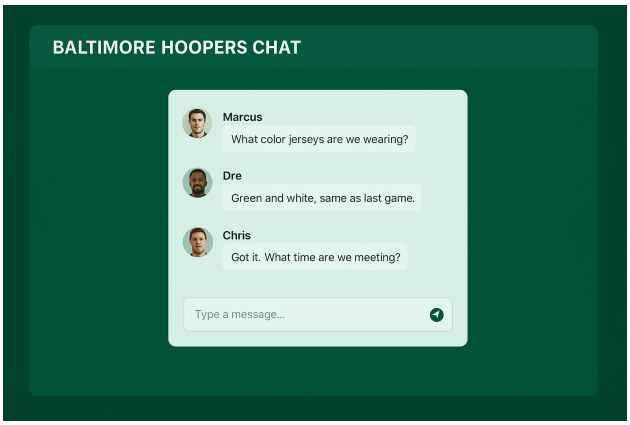


Figure 7: Team Chat Room

5 Iterations

5.1 Iteration Planning

[In CS496, you plan all iterations beforehand. In CS482, you update the planning here at each iteration.]

Iteration	Dates	Stories	Points
1	10/09 - 10/23	T2, A11, M1, U17, Spike20, A14, V5, U18	16
2	10/23 - 11/6	V8 (create + update stats), M20 (add + remove manager), U18 (sign-in/out), T2 (team selection) U19 (view team roster), U16 (set competiton rules)	16
Total:	16		

Table 3: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

Overall, we wanted to focus on the basics of the application, which included creating an account, creating a team, joining a team, and setting up sponsor programs. These user stories would lead to us creating the basics of the application, and get a good handle on how our codebase would look like.

5.2.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

Manuel worked on the tournament creation/reading and team creation/reading. He was able to setup the basic schema of the two, and the routes for the endpoints required to create/read all/read one. However, he did not yet include authentication, as only parent accounts should be able to create teams, and only admin accounts should be able to create tournaments.

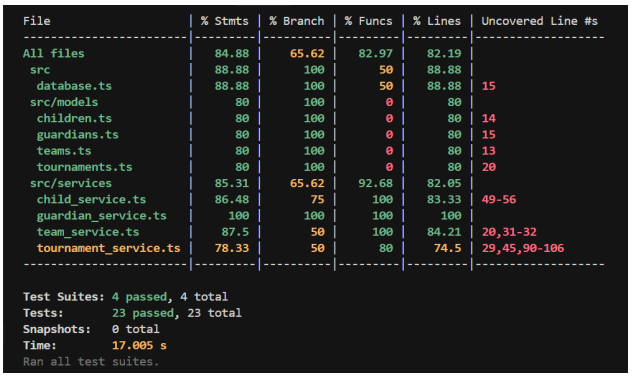
Aidan created an App Bar at the top of the app where users can navigate to different pages. He also created the sign-up pages for guardian, teen, and child accounts. In addition, he enhanced the registration

flow with multi-step form handling, input validation, and snackbar notifications for successful submissions. He refined the user experience by adding age validation based on date of birth input and improving form navigation and error handling for a smoother sign-up process.

Hans: worked on figuring out how to implement the calendar and add indicators under the dates that had events. This included implementing automated dates for each game in the scheduling algorithm. I also worked on the team joining page, showing all available teams with a join button. If a team's button is clicked, updating the team's roster and the player's team id in the database. This is a dummy player for now.

5.2.3 Testing Coverage

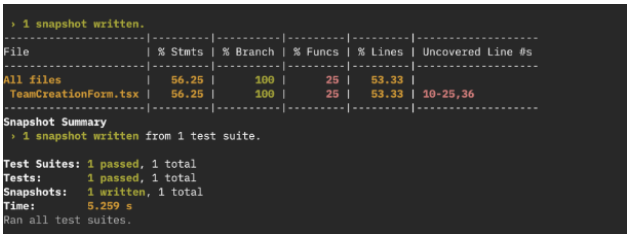
[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]



File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	84.88	65.62	82.97	82.19	
src	88.88	100	50	88.88	
database.ts	88.88	100	50	88.88	15
src/models	80	100	0	80	
children.ts	80	100	0	80	14
guardians.ts	80	100	0	80	15
teams.ts	80	100	0	80	13
tournaments.ts	80	100	0	80	20
src/services	85.31	65.62	92.68	82.05	
child_service.ts	86.48	75	100	83.33	49-56
guardian_service.ts	100	100	100	100	
team_service.ts	87.5	50	100	84.21	20,31-32
tournament_service.ts	78.33	50	80	74.5	29,45,90-106

Test Suites: 4 passed, 4 total
Tests: 23 passed, 23 total
Snapshots: 0 total
Time: 17.005 s
Ran all test suites.

Figure 8: Back-end Coverage



› 1 snapshot written.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	56.25	100	25	53.33	
TeamCreationForm.tsx	56.25	100	25	53.33	10-25,36

Snapshot Summary
› 1 snapshot written from 1 test suite.

Test Suites: 1 passed, 1 total
Tests: 1 passed, 1 total
Snapshots: 1 written, 1 total
Time: 5.259 s
Ran all test suites.

Figure 9: Front-end Coverage

Overall, we should definitely increase both our front-end and back-end coverage.

5.2.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

This iteration, a lot of our individual time went towards a) learning the basics of web-development and b) setting up the code-base. Many of our group members were unfamiliar with all of the parts of web-development, which required time outside of working on the project to tinker around with tutorials and guides unrelated to this project specifically. All of this required learning caused a lot of friction before we could get to actual programming. However, we all learned a lot about the specific libraries we're using. Now that we have a foundation, the next iteration should go smoother.

5.3 Iteration/Sprint 2

5.3.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

In this iteration, we plan to implement user sessions as well as login/logout functionality. This will allow us to fully implement other features that we previously had to use dummy users for. The overall goal for this iteration was to add finishing touches on unfinished features from last iteration and better connect them before moving on to add new features.

5.3.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

Hans: completed login/out with session storage and finishing touches on the event calendar. In hindsight, I completed more points than I thought for the calendar in the first iteration because I implemented automatic date generation for all games in a tournament, which is like a create function. I added the team versus team implementation for the calendar, making it more useful. I also fixed tests for the join team service from last iteration. Manuel worked on creating components to view a player's stats, and update a player's stats for a specific game. Aidan worked on refactoring the "RegistrationForm" component to use the "FormContainer" component. This refactor was made because FormContainer is more widely used throughout the repo. Aidan also added the logic for set competition rules and view competition rules.

5.3.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	65.1	48.52	64.58	61.94	
src	88.88	100	50	88.88	
database.ts	88.88	100	50	88.88	15
src/controllers	63.56	49.24	69.51	59.74	
child_controller.ts	67.03	59.52	94.11	62.33	51-58,67,74,79-80,88,92,99,107-108,123,127,134-154
guardian_controller.ts	67.5	40	61.53	66.66	18-19,24-43
rules_controller.ts	76.92	75	100	72.72	8,16-17,30,41-42
team_controller.ts	83.72	66.66	100	80.55	20,31-32,55,59,67-68
teenager_controller.ts	37.03	31.81	27.27	31.46	...,73-91,97,107-110,118,122,132-138,151,155,162-182
tournament_controller.ts	80.88	50	81.25	77.96	47,72,123-143
src/models	69.23	25	25	69.23	
children.ts	80	100	0	80	15
guardians.ts	73.33	50	50	73.33	17-19,29,35
rules.ts	100	100	100	100	
teams.ts	80	100	0	80	13
teenagers.ts	40	0	0	40	17-19,27-35
tournaments.ts	80	100	0	80	23
src/services	80	100	50	71.42	
hashing.ts	80	100	50	71.42	11-12

Figure 10: Back-end Coverage

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	51.54	6.25	37.03	48.31	
components	53.84	20	33.33	51.61	
StatsEditor.tsx	59.09	0	50	57.14	48-68,95
StatsViewer.tsx	48.14	25	22.22	46.15	37-40,46,52-61,65,79-92
TeamCreationForm.tsx	56.25	100	25	53.33	10-25,36
services	46.87	0	50	40.74	
session_service.ts	66.66	0	75	57.14	15-22,30
team_service.ts	21.42	0	0	23.07	5-33

Figure 11: Front-end Coverage

Overall, our testing is still very low. Ideally, next sprint, we'll prioritize better testing over just getting features made.

5.3.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).] One of the issues the group faced was trying to build a less-standard component, for example an editable data table.

5.4 Iteration/Sprint 3

5.4.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.4.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.4.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.4.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.5 Iteration/Sprint 4

[CS496 has 5 sprints. CS482 only has only 3 sprints (remove Iterations 4 and 5 from this doc if you are writing a doc for 482]

5.5.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.5.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.5.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.5.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.6 Iteration/Sprint 5

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.6.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project.]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently. How can you do better work in future projects? You may write this as a team or per person (or both)]

Appendix

[Appendix section if needed]